Oracle's Assistance

Game Design & Implementation Specification (Revision 2)

Revision note: This revision resolves the open design questions from the first draft. Changes from Revision 1 include: an asynchronous quantum adapter, measurement-collapse behavior after a wrong pot, deferred target generation, a fixed 64-state search space, unlimited offerings in the main loop, defined Zeus verdict thresholds, a fully specified interactive tutorial, centralized dialogue, simple labeled placeholder art, and optional post-MVP audio hooks.

Background

Project page: https://qollab.xyz/programs/hackathon

This project is being made for Qollab's quantum hackathon under the "Quantum Unlocked" theme. It is meant to be educational and should reflect that. The intended audience is competent high schoolers and beyond, not children or professional researchers.

Overall Idea

A simple platformer game that explains Grover's algorithm. Hercules must find his father's lightning bolt, hidden in one of 64 pots. He first tries brute force, then receives help from the Oracle of Delphi, who acts as the bridge into the quantum world. Through talking with the Oracle and using his six magic coins (which represent qubits), Hercules reduces his search from up to 63 classical checks (worst case, $N - 1$) to roughly $\sqrt{N}$ queries: about 6 Grover iterations ($\pi/4 \cdot \sqrt{64} \approx 6.3$) followed by a single pot check.

Story and Core Gameplay

Zeus tells Hercules that Hermes, the trickster god, has stolen his lightning bolt and hidden it inside the Temple of Apollo. Zeus needs it back quickly, and he trusts Hercules, who solved difficult problems throughout his twelve labors. Hercules agrees and makes the short trek to the temple. The trek involves light parkour, and Hercules must bring an offering to the Oracle to gain entrance.

Inside, he sees the wall of 64 pots where Hermes must have hidden the bolt. He guesses one at random and finds it empty. The Oracle sends him out of the temple, telling him to bring another offering to check another pot. He climbs back down, fetches another offering, returns, and fails again. He realizes there must be a better way.

Hercules carries six golden coins. He knows that six two-sided coins can represent all 64 pots as a bit string, but he wonders how to gain information about the bolt's location without opening pots. He asks the Oracle, who explains Grover's algorithm through an interactive tutorial. Hercules decides to use it. From then on he may either ask the Oracle to run one Grover iteration or inspect a pot. Each action costs one offering, symbolizing computational cost. He repeats these as he likes until he finds the bolt. Afterward, the Oracle explains how many actions he saved compared to brute force, and Zeus reacts according to how quickly the bolt was found.

The on-screen name for the Oracle is **"the Oracle of Delphi"** (shortened to "Oracle" in dialogue headers). She is never called "Delphi" as a personal name.

Game Design

Oracle's Assistance is a 16-bit-style 2D vertical platformer, closest in feel to Shovel Knight or Terraria. The player makes only two kinds of decisions: movement and jumping during the parkour, and whether to inspect a pot or run another Grover iteration.

Opening the webpage shows a title card with a Play button. The game begins with Hercules idling on the hill below the temple, without player control. Zeus floats down, and a scroll-style dialogue window delivers the exposition. The player must click through all of it, after which control begins.

The hill is a flat area spanning the screen, with sparse decoration. A pile of offerings sits at its center. Interacting with the pile (E) collects one offering, with a maximum of one carried at a time. The offering is consumed on entering the temple, and a short message tells the player so. Movement feels responsive and smooth, with industry-standard acceleration, coyote time, and jump buffering. There is no fall damage and no punishing mechanic. The climb should be fun but take no longer than 30 seconds.

On the first two visits, Hercules can only break a pot, and he is prompted to do so both times. Only on the third visit does the Oracle conversation and tutorial occur. After the tutorial, a permanent on-screen button reopens the Grover explanation at any time. When the bolt is found, Zeus descends once more and thanks Hercules (or scolds him), and the game ends on a results screen showing total actions, with pot checks and Oracle queries each counting as one.

Visuals

The setting is a sunny seaside region of ancient Greece. The final art direction resembles Terraria: simple 16-bit foreground sprites and slightly more detailed, still pixelated, parallaxed background set pieces. The temple floats in the sky above the hill. It looks classically Greek and, like a Terraria house, has background walls but functions as an open 2D space, with real walls and ceiling in the foreground. Platforms resemble Greek marble columns and ledges. Fonts are legible but evoke ancient Greece. Dialogue windows resemble scrolls.

MVP art policy: placeholders only. Final visuals will be produced separately. The build must ship with simple, correctly sized placeholders: solid colored rectangles with a 1 px darker outline and a short text label (e.g., "HERC", "ZEUS", "ORACLE", "POT 37"). Animation frames for placeholders may be identical or use minimal variation (e.g., a 1–2 px bob) so that animation states are visibly working. Every visual is loaded by a fixed key and file path; dropping a correctly named PNG/spritesheet into the asset folder must replace the placeholder with no code changes.

Quantum Implementation

There are two quantum components, both to run within Qollab's software and rules in the final version:

- **Target selection:** the bolt's location is chosen by a quantum random number generator.
- **Grover's algorithm:** a genuine Grover implementation is invoked each time the player asks the Oracle.

The test/MVP version does not use Qollab software. It runs a classical simulation behind the same interface. The real quantum implementations are written separately and are out of scope for this build, but the simulation is in scope.

Quantum State Rules

- **Fixed search space:** Grover always operates over all 64 states (6 qubits). Broken pots are not removed from the search space; this keeps the $2^6 = 64$ story consistent.
- **Measurement collapse:** Inspecting a pot is a measurement. If the pot is wrong, the Grover state resets to uniform superposition and the current-run iteration counter returns to 0. The Oracle explains this briefly the first time it happens. The HUD's total Grover iteration count does not reset.
- **Over-rotation is allowed:** The player may run more iterations than optimal; the probability of the target then decreases, as it would on real hardware.
- **Deferred target generation:** The target is generated exactly once per playthrough, at the start of the main loop (after the tutorial), and excludes the two pots broken in the forced attempts. This guarantees both early attempts fail without rigging any later outcome.

Implementation Specification

Build as a browser-based 2D platformer using **Phaser 3** with standard HTML/CSS/JavaScript (ES modules). No backend; it runs locally through a simple static server (e.g., `python -m http.server`). Phaser is vendored locally in the project so the game works offline for judging. Target desktop Chrome first. Fixed internal resolution 960×540 (16:9) with responsive scaling, nearest-neighbor rendering, and antialiasing disabled.

Controls

- A / Left Arrow: move left
- D / Right Arrow: move right
- Space / W / Up Arrow: jump
- E: interact
- Mouse: dialogue and menu selection
- Esc: close optional menus

Movement Tuning

Responsive and slightly arcade-like, not floaty. Approximate values:

Maximum horizontal speed	220 px/s
Ground acceleration	1600 px/s²
Ground deceleration	1900 px/s²
Air acceleration	1100 px/s²
Gravity	1100 px/s²
Jump velocity	–430 px/s
Coyote time	100 ms
Jump buffer	120 ms

Variable jump height	Cut upward velocity when jump is released early

No combat, health, lives, fall damage, double jump, wall jump, dash, inventory system, enemies, or other unrequested mechanics. Falling respawns Hercules at the most recent safe checkpoint.

Level Structure

The world is about 960×2200 px tall. Hercules starts on the hill at the bottom; the Temple of Apollo is at the top. The ascent uses 14–18 marble platforms/columns, with no required jump exceeding about 80% of Hercules's maximum range. A competent player reaches the temple in 20–30 seconds. The camera follows vertically with slight smoothing. Parallax factors:

- Sky: effectively fixed
- Very far scenery/clouds: ~0.10×
- Distant coastline/mountains: ~0.20×
- Near scenery: ~0.40×
- Gameplay layer: 1.00×

Game Progression (Strict State Machine)

1. **Introduction:** Player control disabled. Zeus descends, delivers the task, and ascends. Control begins.
2. **First Attempt:** Hercules collects one offering, climbs to the temple, the offering is consumed, and he is prompted to inspect a pot via the 8×8 grid. The pot is empty (guaranteed, since no target exists yet). He is returned to the hill.
3. **Second Attempt:** The same process repeats with the same guarantee.
4. **Oracle Introduction:** On the third visit, the Oracle dialogue and the interactive Grover tutorial run.
5. **Main Game Loop:** The target is generated now (excluding the two broken pots). Platforming is no longer required; the player stays in the temple with an **unlimited** supply of offerings. Each action costs exactly one offering/action: run one Grover iteration, or inspect one pot. The player alternates freely until the bolt is found.
6. **Ending:** Finding the bolt plays a reveal animation, Zeus returns, the Oracle summarizes actions saved versus brute force, Zeus gives his verdict, and the results screen appears.

Zeus's Verdict

Counted over main-loop actions (Grover iterations + pots inspected after the tutorial). The two forced early pots are shown on the results screen but excluded from the verdict.

Main-loop actions	Zeus's reaction
10 or fewer	Pleased
11 to 20	Neutral / impatient
More than 20	Angry

The results screen shows: total actions, Grover iterations used, pots inspected, Zeus's verdict, and a comparison to the classical expected (~32) and worst case (63). A Play Again button starts a completely

fresh run.

Temple and Pots

Exactly 64 pots in an 8×8 grid, corresponding to six-bit states 000000 through 111111. The player does not walk between pots; choosing to inspect opens an interactive 8×8 selection interface. Pots highlight on hover, display their decimal index and six-bit representation, remain visibly broken after inspection, and become unselectable once checked. A wrong pot plays a short break animation and shows an empty result. The correct pot reveals Zeus's lightning bolt.

Grover Visualization

After each iteration, show the probability distribution across all 64 states as an 8×8 grid whose brightness corresponds to probability. It is not a direct reveal of the target. The display also shows the current-run iteration count and the probability of the most likely state. The interface must make clear that Grover's algorithm raises the probability of the correct state but does not simply reveal the answer, and a short note should state that real quantum hardware only yields a measured outcome. The visible distribution is the Oracle's "vision," shown for teaching purposes.

Interactive Grover Tutorial

Four concise stages, navigable with Next/Back. After it is completed once, a permanent "Grover Explanation" button reopens it at any time (Esc closes it).

1. **Encoding:** Six clickable coins flip between 0 and 1; a readout shows the binary string, its decimal value, and the corresponding pot highlighted on a mini 8×8 grid. Message: six qubits represent $2^6 = 64$ possibilities.
2. **Superposition:** A button applies "Hadamard to all coins"; 64 amplitude bars become equal (1/64 probability each).
3. **Oracle:** A "Mark target" button flips the phase of one demo state; its bar inverts below the axis while its probability stays the same. Message: the oracle marks the answer without revealing it.
4. **Amplification:** A step button applies diffusion; bars reflect about the mean and the marked state grows. The player can step through roughly 10 iterations and watch the success probability peak near iteration 6 and then fall (over-rotation).

The tutorial uses its own demo target, never the real one.

Quantum/Game Interface

All quantum functionality is isolated in `src/quantumAdapter.js` so the classical simulation can be replaced without touching gameplay code. Every function is **asynchronous** (returns a Promise) to accommodate real quantum calls with network or hardware latency:

```
async generateTarget(excludedIndices: number[]) -> number   // 0-63
async runGroverIteration() -> {
  iteration: number,            // current-run iteration count
  probabilities: number[64],    // sums to 1
  mostLikely: number            // index of highest probability
}
async resetGrover() -> void     // return to uniform superposition
```

While any adapter call is pending, the game shows an "The Oracle is consulting..." state and blocks further actions, preventing duplicate requests. The game itself never contains quantum logic.

HUD

During gameplay show: whether Hercules carries an offering (platforming phases only), total actions, pots checked, and Grover iterations used. Once unlocked, also show the Grover Explanation button.

Dialogue

All dialogue text lives in a single file, `src/dialogue.js`, so it can be edited without touching game logic. The tone is accurate but light, written for high schoolers. Dialogue pauses player movement where appropriate. Each line shows the speaker's portrait (placeholder) and name in a scroll-styled window.

Characters

- **Hercules:** idle, run, jump/fall, interact; left/right facing.
- **Zeus:** floating idle, simple descent/ascent.
- **Oracle:** stationary idle, dialogue interaction.

Each has a close-up dialogue portrait (placeholder for MVP).

Assets and File Structure

```
/index.html
/lib/phaser.min.js
/src/main.js, scenes/, quantumAdapter.js, dialogue.js, assetManifest.js
/assets/characters/  /assets/portraits/  /assets/environment/
/assets/background/  /assets/ui/         /assets/audio/
```

`assetManifest.js` lists every asset key, path, and expected size. If a file is missing, a correctly sized labeled placeholder is generated at load time. Suggested sizes: Hercules 32×48 per frame; Oracle 32×48 per frame; Zeus 48×64 per frame; environment tiles 16×16 or 32×32; portraits 96×96.

Audio (Post-MVP)

Audio is not required for the MVP and the game is silent by default. The build includes a small audio wrapper with named cues (e.g., *jump, offering_pickup, pot_break, bolt_reveal, oracle_query, dialogue_advance, music_hill, music_temple*) listed in the asset manifest. Calls to missing sounds are silent no-ops, so adding files to `/assets/audio/` later enables sound with no gameplay-code changes. A mute toggle may be added at that time.

Required Robustness

- Complete a full playthrough without console errors
- Prevent duplicate pot interactions and duplicate in-flight Oracle queries
- Prevent temple entry without an offering
- Prevent carrying multiple offerings
- Preserve broken-pot state
- Generate the target exactly once per playthrough

- Prevent dialogue/menu softlocks
- Pause movement during forced dialogue
- Restart cleanly from the ending screen
- Work fully with placeholder assets and no audio files
- Contain no mechanics beyond those explicitly specified

Priority goes to completing and polishing this exact gameplay loop rather than adding features.

Build Phases

1 Project skeleton, asset/placeholder system, movement tuning, level, parallax

2 State machine, dialogue system, offerings, forced attempts

3 Pot grid, quantum adapter (classical sim), Grover visualization, tutorial

4 Ending, HUD, results screen, robustness pass, polish